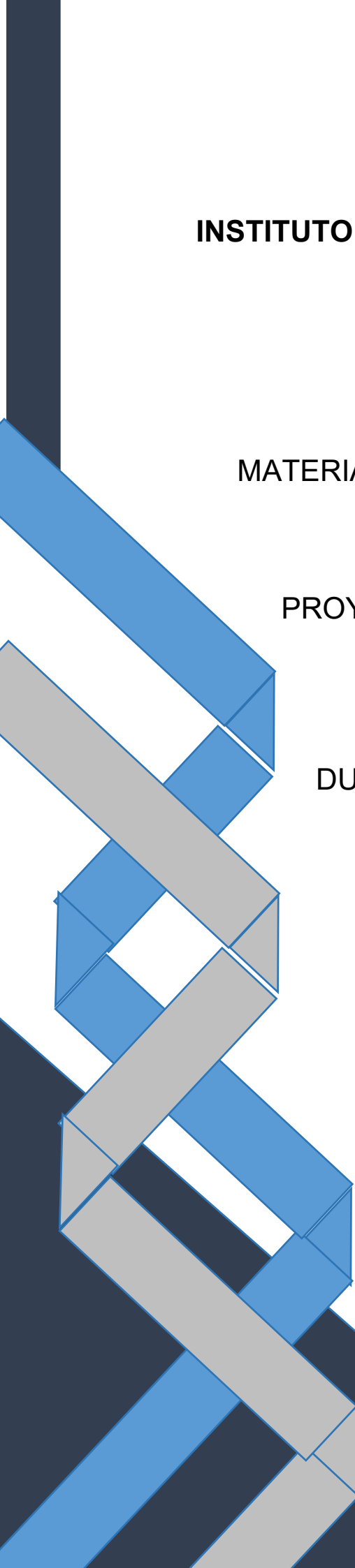




INSTITUTO TECNOLOGICO SUPERIOR DE LERDO

**MATERIA: DESARROLLO DE APLICACIONES
MOVILES**

PROYECTO DE EVIDENCIAS CORTE 3

ALUMNO:

DULCE MARIA REYES MARTINEZ

222310922

INGENIERÍA INFORMÁTICA

DOCENTE:

ING JESUS SALAS

Mi primera aplicacion en flutter

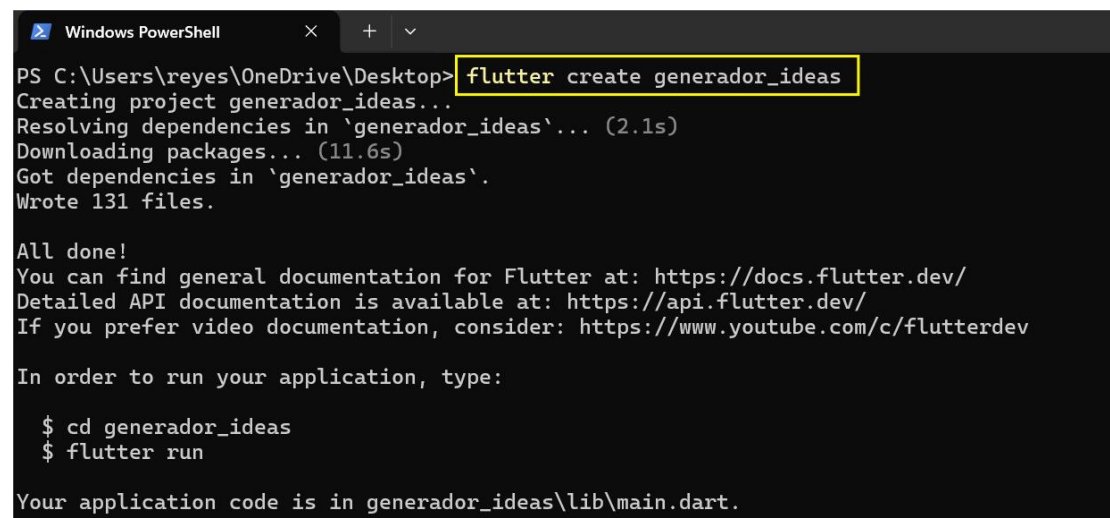
Introduccion

En esta práctica desarrollé una aplicación en Flutter desde cero, con el objetivo de comprender cómo funciona la creación de interfaces, el manejo del estado y la navegación entre pantallas.

La aplicación genera palabras aleatorias, permite marcarlas como favoritas y navegar entre una pantalla principal y una de favoritos.

1. Creación del proyecto

Primero creé el proyecto en Flutter usando el siguiente comando:



```
Windows PowerShell
PS C:\Users\reyes\OneDrive\Desktop> flutter create generador_ideas
Creating project generador_ideas...
Resolving dependencies in 'generador_ideas'... (2.1s)
Downloading packages... (11.6s)
Got dependencies in 'generador_ideas'.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd generador_ideas
$ flutter run

Your application code is in generador_ideas\lib\main.dart.
```

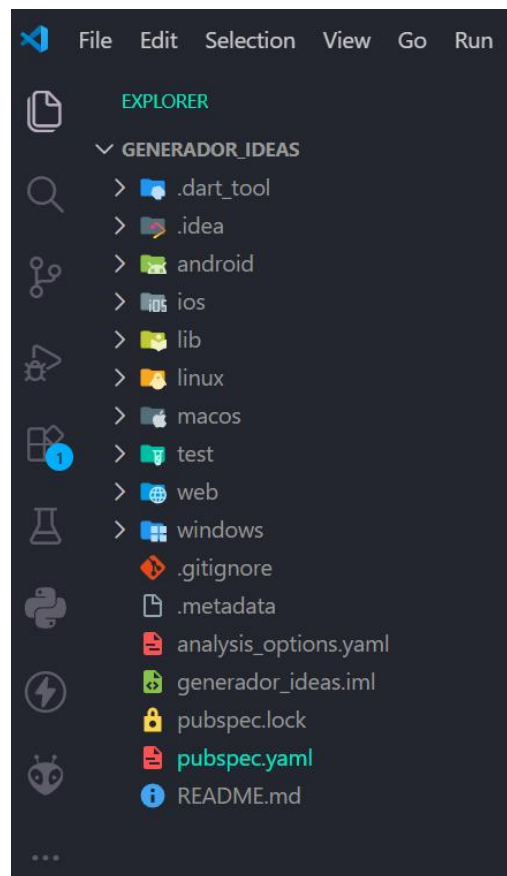
Después de crear el proyecto, es necesario ingresar a la carpeta generada desde la terminal. Para ello, se utiliza el siguiente comando:



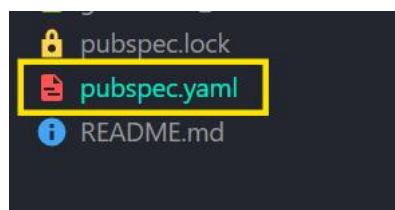
```
PS C:\Users\reyes\OneDrive\Desktop> cd generador_ideas
```

Una vez dentro de la carpeta del proyecto, se procede a abrirlo en Visual Studio Code con el siguiente comando:

```
PS C:\Users\reyes\OneDrive\Desktop\generador_ideas> code .
```



En el archivo pubspec.yaml, dentro de la sección de dependencias, se deben agregar las siguientes librerías:



- english_words: ^4.0.0
- provider: ^6.0.0

En el archivo pubspec.yaml, dentro de la sección de dependencias, agregué las librerías english_words y provider, ya que son necesarias para el funcionamiento de la aplicación.

La librería english_words se utiliza para generar palabras en inglés de manera aleatoria, lo cual es útil para mostrar ejemplos o contenido dinámico dentro de la app.

Por otro lado, provider se usa para manejar el estado de la aplicación, permitiendo compartir datos entre diferentes partes de forma más sencilla y organizada.

Con estas dependencias, la aplicación puede generar contenido dinámico y mantener una mejor estructura en el manejo de la información.

```
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  english_words: ^4.0.0
  provider: ^6.0.0
```

Después ajusté el archivo de configuración `analysis_options.yaml` para que el análisis del código fuera menos estricto, ya que es una práctica inicial.

Archivo: `analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    prefer_const_constructors: false
    avoid_print: false
```

Este archivo define las reglas que Flutter utiliza para analizar el código, y en esta práctica se configuró para que sea menos estricto, permitiendo trabajar de manera más sencilla sin recibir demasiadas advertencias, lo cual es útil cuando se está aprendiendo.

II. Estructura de la aplicación

La aplicación se construye a partir de widgets, que son los elementos principales en Flutter.

Primero se define la función `main()`, que es la encargada de iniciar la app. Después, se crea la clase `MyApp`, la cual configura el tema, el nombre de la aplicación y el widget principal.

Para manejar los datos, utilicé una clase llamada `MyAppState`, la cual guarda:

- ◆ La palabra actual generada
- ◆ Una lista de palabras favoritas

Esta clase usa ChangeNotifier, lo que permite actualizar automáticamente la interfaz cuando cambian los datos.

Explicación por partes del código principal

main.dart

Importaciones

Las importaciones permiten agregar funcionalidades externas al archivo donde se está trabajando. En este proyecto se importaron librerías como material.dart para utilizar los widgets y el diseño visual de Flutter, english_words para generar palabras aleatorias y provider para manejar el estado de la aplicación. Estas importaciones son necesarias para poder utilizar todas las herramientas y funciones que requiere la aplicación.

```
1  import 'package:english_words/english_words.dart';
2  import 'package:flutter/material.dart';
3  import 'package:provider/provider.dart';
4
```

Funcion Main

La función main() es el punto de inicio de toda la aplicación, ya que es la primera función que se ejecuta cuando el programa comienza. Dentro de ella se utiliza runApp() para cargar el widget principal de la aplicación, que en este caso es MyApp. Gracias a esta función, Flutter puede iniciar correctamente toda la interfaz y el funcionamiento de la app.

```
5  void main() {
6    runApp(MyApp());
7  }
```

Clase MyApp

En esta parte de la aplicación utilicé la clase MyApp para configurar toda la aplicación de manera general. Dentro de esta clase definí el tema visual, la estructura principal y la pantalla que se mostrará al iniciar la aplicación. También utilicé ChangeNotifierProvider para compartir el estado de la aplicación entre los diferentes widgets, permitiendo que todos puedan acceder a la información y reaccionar automáticamente cada vez que exista algún cambio en los datos.

```
9  class MyApp extends StatelessWidget {
10    const MyApp({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      return ChangeNotifierProvider(
15        create: (context) => MyAppState(),
16        child: MaterialApp(
17          debugShowCheckedModeBanner: false,
18          title: 'Wordify',
19          theme: ThemeData(
20            useMaterial3: true,
21            colorScheme: ColorScheme.fromSeed(
22              seedColor: Colors.deepOrange,
23            ), // ColorScheme.fromSeed
24          ), // ThemeData
25          home: MyHomePage(),
26        ), // MaterialApp
27      ); // ChangeNotifierProvider
28    }
29  }
```

Clase MyAppState

En esta parte de la aplicación utilicé la clase `MyAppState` para manejar toda la información dinámica que necesita la aplicación. Dentro de esta clase almacené la palabra generada actualmente y también la lista de palabras favoritas del usuario. Además, implementé métodos como `getNext()`, el cual se encarga de generar nuevas palabras aleatorias, y `toggleFavorite()`, que permite agregar o eliminar palabras de la lista de favoritos según la acción realizada por el usuario. Cada vez que ocurre un cambio en la información, la interfaz se actualiza automáticamente utilizando `notifyListeners()`, permitiendo que los cambios se reflejen de inmediato en la aplicación.

```
31  class MyAppState extends ChangeNotifier {
32      var current = WordPair.random();
33
34      var favorites = <WordPair>[];
35
36      void getNext() {
37          current = WordPair.random();
38
39          notifyListeners();
40      }
41
42      void toggleFavorite() {
43          if (favorites.contains(current)) {
44              favorites.remove(current);
45          } else {
46              favorites.add(current);
47          }
48
49          notifyListeners();
50      }
51  }
```


Clase MyHomePage (StatefulWidget)

Utilicé la clase MyHomePage como un StatefulWidget porque necesitaba manejar cambios dinámicos dentro de la interfaz. Esto me permitió controlar la navegación entre las diferentes pantallas de la aplicación, cambiando entre la página principal y la sección de favoritos según la opción seleccionada por el usuario.

```
53  class MyHomePage extends StatefulWidget {  
54      @override  
55      State<MyHomePage> createState() => _MyHomePageState();  
56  }  
57  
58  class _MyHomePageState extends State<MyHomePage> {
```

Variable selectedIndex

También utilicé la variable selectedIndex para guardar el índice de la pantalla que se encuentra activa en el menú de navegación, ya que dependiendo del valor almacenado en esta variable se muestra una pantalla diferente, permitiendo alternar fácilmente entre las distintas secciones de la aplicación.

```
59      int selectedIndex = 0;  
60  
61      @override  
62      Widget build(BuildContext context) {  
63          Widget page;  
64
```

Estructura switch para navegación

El bloque switch se utilizó para controlar qué pantalla se mostraría dependiendo del valor que tuviera la variable selectedIndex. Gracias a esto, la aplicación puede cambiar entre la pantalla principal llamada GeneratorPage y la pantalla de favoritos llamada FavoritesPage. Esta estructura permitió realizar la navegación dentro de la aplicación de una forma más organizada y sencilla.

```
65     switch (selectedIndex) {
66     case 0:
67         page = GeneratorPage();
68         break;
69
70     case 1:
71         page = FavoritesPage();
72         break;
73
74     default:
75         throw UnimplementedError(
76             'No widget for $selectedIndex',
77         );
78     }
79
80     return LayoutBuilder(
81         builder: (context, constraints) {
82             return Scaffold(
83                 body: Row(
84                     children: [
```

NavigationRail y Scaffold

El widget `NavigationRail` se utilizó para crear un menú lateral con las diferentes opciones de navegación de la aplicación. Por otra parte, `Scaffold` se encargó de organizar toda la estructura visual de la interfaz, permitiendo acomodar correctamente los elementos en pantalla. Ambos trabajan en conjunto para que, al seleccionar una opción del menú, la interfaz cambie utilizando `setState()`, actualizando automáticamente la pantalla mostrada.

```
return Scaffold(  
  body: Row(  
    children: [  
      SafeArea(  
        child: NavigationRail(  
          extended: constraints.maxWidth >= 600,  
          destinations: [  
            NavigationRailDestination(  
              icon: Icon(Icons.home),  
              label: Text('Home'),  
            ),  
            NavigationRailDestination(  
              icon: Icon(Icons.favorite),  
              label: Text('Favorites'),  
            ),  
          ],  
          selectedIndex: selectedIndex,  
          onDestinationSelected: (value) {  
            setState(() {  
              selectedIndex = value;  
            });  
          },  
        ),  
      ),  
      Expanded(  
        child: Container(  
          color: Theme.of(context)  
            .colorScheme  
            .primaryContainer,  
          child: page,  
        ),  
      ),  
    ],  
  ),  
);  
};  
};  
};  
}
```

Clase GeneratorPage

La clase `GeneratorPage` se encargó de representar la pantalla principal de la aplicación. En esta parte se muestra la palabra generada junto con los botones que permiten interactuar con ella. Además, esta clase utiliza el estado global para obtener la palabra actual y permite tanto generar nuevas palabras como agregarlas a favoritos mediante los botones disponibles.



```
1  class GeneratorPage extends StatelessWidget {
2    @override
3    Widget build(BuildContext context) {
4      var appState = context.watch<MyAppState>();
5
6      var pair = appState.current;
7
8      IconData icon;
9
10     if (appState.favorites.contains(pair)) {
11       icon = Icons.favorite;
12     } else {
13       icon = Icons.favorite_border;
14     }
15
16     return Center(
17       child: Column(
18         mainAxisAlignment: MainAxisAlignment.center,
19         children: [
20           BigCard(pair: pair),
21
22           SizedBox(height: 10),
23
24           Row(
25             mainAxisAlignment: MainAxisAlignment.min,
26             children: [
```

Botones (Like y Next)

Los botones fueron utilizados para permitir la interacción del usuario con la aplicación. El botón "Like" se encarga de llamar al método `toggleFavorite()`, el cual permite guardar o eliminar palabras de la lista de favoritos. Mientras tanto, el botón "Next" ejecuta el método `getNext()`, cuya función es generar una nueva palabra automáticamente y actualizar la interfaz en pantalla.



```

1  ElevatedButton.icon(
2      onPressed: () {
3          appState.toggleFavorite();
4      },
5
6      icon: Icon(icon),
7
8      label: Text('Like'),
9  ),
10
11  SizedBox(width: 10),
12
13  ElevatedButton(
14      onPressed: () {
15          appState.getNext();
16      },
17
18      child: Text('Next'),
19  ),
20 ],
21 ),
22 ],
23 ),
24 );
25 }
26 }

```

Clase BigCard

La clase BigCard se utilizó para mostrar la palabra generada de una manera más visual y atractiva para el usuario. Para ello, se implementó un diseño basado en tarjetas utilizando el widget Card, además de aplicar colores y estilos de texto más grandes para mejorar la apariencia y la experiencia visual dentro de la aplicación.

```

1  class BigCard extends StatelessWidget {
2    const BigCard({
3      super.key,
4      required this.pair,
5    });
6
7    final WordPair pair;
8
9    @override
10   Widget build(BuildContext context) {
11     final theme = Theme.of(context);
12
13     final style = theme.textTheme.displayMedium!.copyWith(
14       color: theme.colorScheme.onPrimary,
15     );
16
17     return Card(
18       color: theme.colorScheme.primary,
19
20       elevation: 8,
21
22       child: Padding(
23         padding: const EdgeInsets.all(20),
24
25         child: Text(
26           pair.asLowerCase,
27
28           semanticsLabel:
29             "${pair.first} ${pair.second}",
30
31           style: style,
32         ),
33       ),
34     );
35   }
36 }

```

Clase FavoritesPage

La clase FavoritesPage representa la pantalla donde se almacenan y muestran todas las palabras favoritas que el usuario haya guardado. Cuando no existen elementos guardados, se muestra un mensaje indicando que no hay favoritos disponibles. En cambio, cuando sí existen palabras almacenadas, estas se presentan en una lista utilizando ListView para organizarlas correctamente en pantalla.

```

1  class FavoritesPage extends StatelessWidget {
2    @override
3    Widget build(BuildContext context) {
4      var appState = context.watch<MyAppState>();
5
6      if (appState.favorites.isEmpty) {
7        return Center(
8          child: Text(
9            'No hay favoritos todavía.',
10          ),
11        );
12      }
13
14      return ListView(
15        children: [
16          Padding(
17            padding: const EdgeInsets.all(20),
18
19            child: Text(
20              'Tus favoritos:',
21              style: TextStyle(
22                fontSize: 24,
23                fontWeight: FontWeight.bold,
24              ),
25            ),
26          ),
27
28          for (var pair in appState.favorites)
29            ListTile(
30              leading: Icon(
31                Icons.favorite,
32                color: Colors.red,
33              ),
34
35              title: Text(pair.asLowerCase),
36            ),
37        ],
38      );
39    }
40  }

```

Ejecución de la aplicación

Para ejecutar la aplicación utilicé la terminal de Visual Studio Code y el navegador Microsoft Edge como dispositivo de prueba. Primero abrí la terminal dentro del proyecto Flutter y ejecuté el siguiente comando:

```
flutter run -d edge
```



```
PS C:\Users\reyes\OneDrive\Desktop\generador_ideas> flutter run -d edge
Launching lib\main.dart on Edge in debug mode...
Waiting for connection from debug service on Edge... 38.9s

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

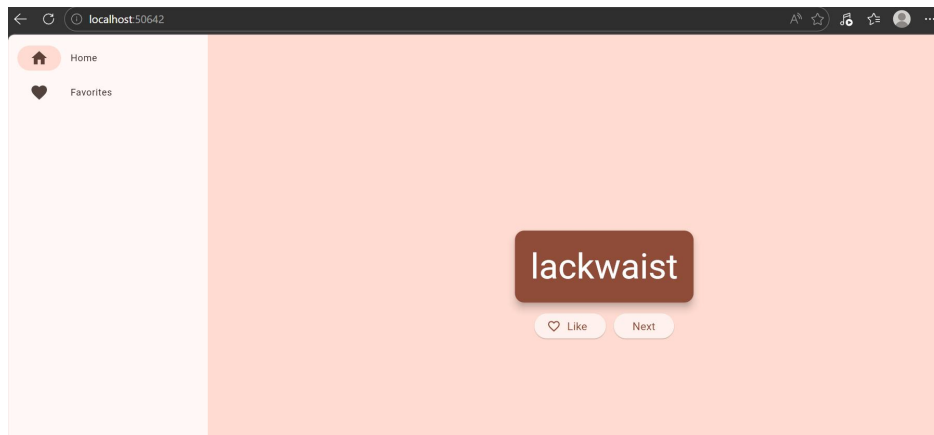
Debug service listening on ws://127.0.0.1:60700/XjYbwKyoxYU=/ws
A Dart VM Service on Edge is available at: http://127.0.0.1:60700/XjYbwKyoxYU=
The Flutter DevTools debugger and profiler on Edge is available at:
```

Este comando permitió compilar y ejecutar la aplicación en el navegador Edge utilizando Flutter Web. Durante la ejecución, Flutter mostró mensajes indicando que la aplicación se estaba compilando correctamente y que el servicio de depuración había iniciado.

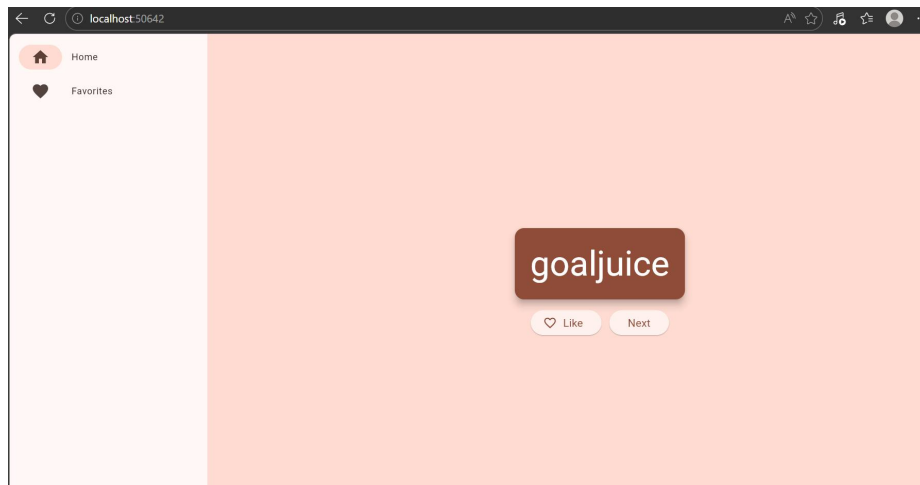
La terminal mostró información como la conexión del servicio de depuración, comandos disponibles para recarga rápida (Hot Reload) y la dirección local donde se estaba ejecutando la aplicación.

Posteriormente, la aplicación abrió automáticamente una ventana en Microsoft Edge mostrando la interfaz principal del proyecto. En la pantalla se pudo observar:

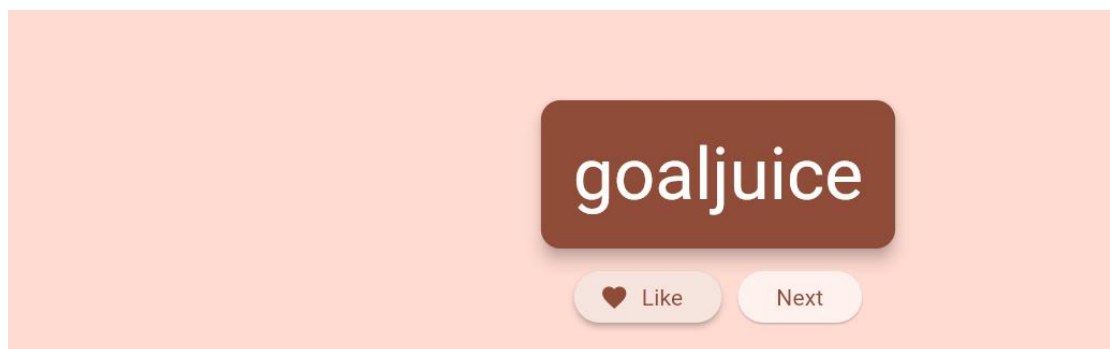
- ✓ Un menú lateral con las opciones Home y Favorites
- ✓ Una tarjeta central mostrando palabras aleatorias
- ✓ Un botón Like para guardar favoritos
- ✓ Un botón Next para generar nuevas palabras
- ✓ La aplicación funcionó correctamente, permitiendo generar palabras nuevas y guardar elementos favoritos sin errores.



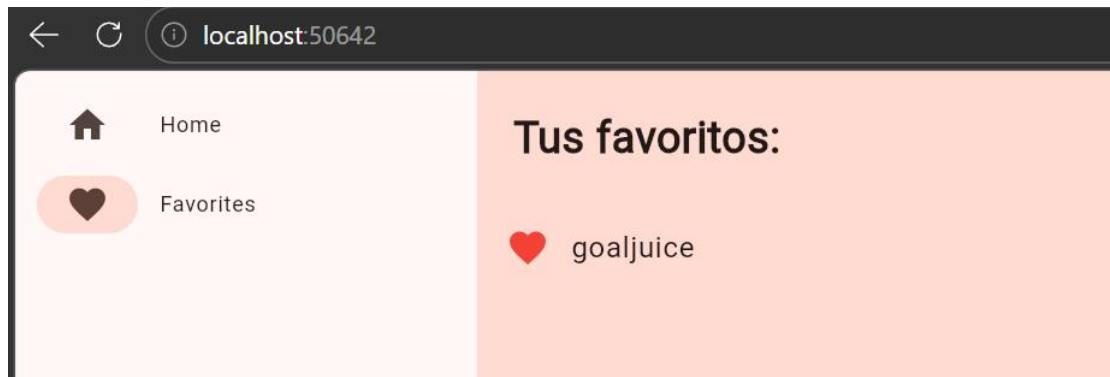
Se pueden generar palabras dandole al boton next



Y la palabra que le des like se guarda



En la apartado de tus favoritos



Conlusion

En esta práctica aprendí a crear una aplicación básica en Flutter utilizando widgets, navegación y manejo de estado. También comprendí cómo generar palabras aleatorias, guardar elementos favoritos y organizar la interfaz de manera más visual e interactiva. Además, aprendí a ejecutar la aplicación en Microsoft Edge usando Flutter Web y a utilizar herramientas como Provider para actualizar la interfaz automáticamente. Esta práctica me ayudó a entender mejor cómo funciona el desarrollo de aplicaciones móviles y web con Flutter.